

OS LAB-6
DATE: 13/4/26

SUMANT SHRIDHAR
USN: 1BM24CS299
BATCH: 4K2

1) Earliest Deadline First

```
#include <stdio.h>
#include <limits.h>

#define MAX 20

struct Task {
    int id;
    int at;
    int C;
    int D;
    int remaining;
};

int main() {
    int n, i, time = 0, completed = 0, max_time;
    float U = 0;

    struct Task t[MAX];

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        t[i].id = i + 1;

        printf("Task %d Arrival Time: ", i + 1);
        scanf("%d", &t[i].at);
        printf("Task %d Execution Time: ", i + 1);
        scanf("%d", &t[i].C);
        printf("Task %d Deadline: ", i + 1);
        scanf("%d", &t[i].D);

        t[i].remaining = t[i].C;
        U += (float)t[i].C / t[i].D;
    }

    printf("Enter total simulation time: ");
    scanf("%d", &max_time);

    if (U > 1) printf("Scheduling not feasible (deadlines may be missed)\n");
    else printf("Scheduling feasible\n");

    printf("\nTime\tTask\n");
```

```

while (time < max_time && completed < n) {
    int idx = -1;
    int min_deadline = INT_MAX;
    for (i = 0; i < n; i++) {

        if (t[i].at <= time && t[i].remaining > 0) {

            int abs_deadline = t[i].at + t[i].D;

            if (abs_deadline < min_deadline) {
                min_deadline = abs_deadline;
                idx = i;
            }
        }
    }

    if (idx == -1) {
        printf("%d\tIdle\n", time);
    } else {
        printf("%d\tT%d\n", time, t[idx].id);
        t[idx].remaining--;

        if (t[idx].remaining == 0) {
            completed++;
        }
    }
    time++;
}
return 0;
}

```

D:\IBM24CS299\EDF2.exe

```
Enter number of tasks: 3
Task 1 Arrival Time: 0
Task 1 Execution Time: 1
Task 1 Deadline: 4
Task 2 Arrival Time: 1
Task 2 Execution Time: 1
Task 2 Deadline: 3
Task 3 Arrival Time: 2
Task 3 Execution Time: 2
Task 3 Deadline: 6
Enter total simulation time: 10
Scheduling feasible
```

Time	Task
0	T1
1	T2
2	T3
3	T3

```
Process returned 0 (0x0)   execution time : 12.300 s
Press any key to continue.
```


2) Proportional Share Scheduling

```
#include <stdio.h>

#define MAX 20
#define TIME_QUANTUM 10

struct Process {
    int id;
    int bt;
    int remaining;
    int weight;
};

int main() {
    int n, i, time = 0, completed = 0;
    int total_weight = 0;

    struct Process p[MAX];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {

        p[i].id = i + 1;
        printf("P%d Burst Time: ", i + 1);
        scanf("%d", &p[i].bt);
        printf("P%d Weight: ", i + 1);
        scanf("%d", &p[i].weight);
        p[i].remaining = p[i].bt;
        total_weight += p[i].weight;
    }

    printf("\nTime\tProcess\n");

    while (completed < n) {
        if (total_weight == 0) break;
        for (i = 0; i < n; i++) {
            if (p[i].remaining > 0) {

                int slice = (int)((float)p[i].weight / total_weight * TIME_QUANTUM);
                if (slice <= 0) slice = 1;

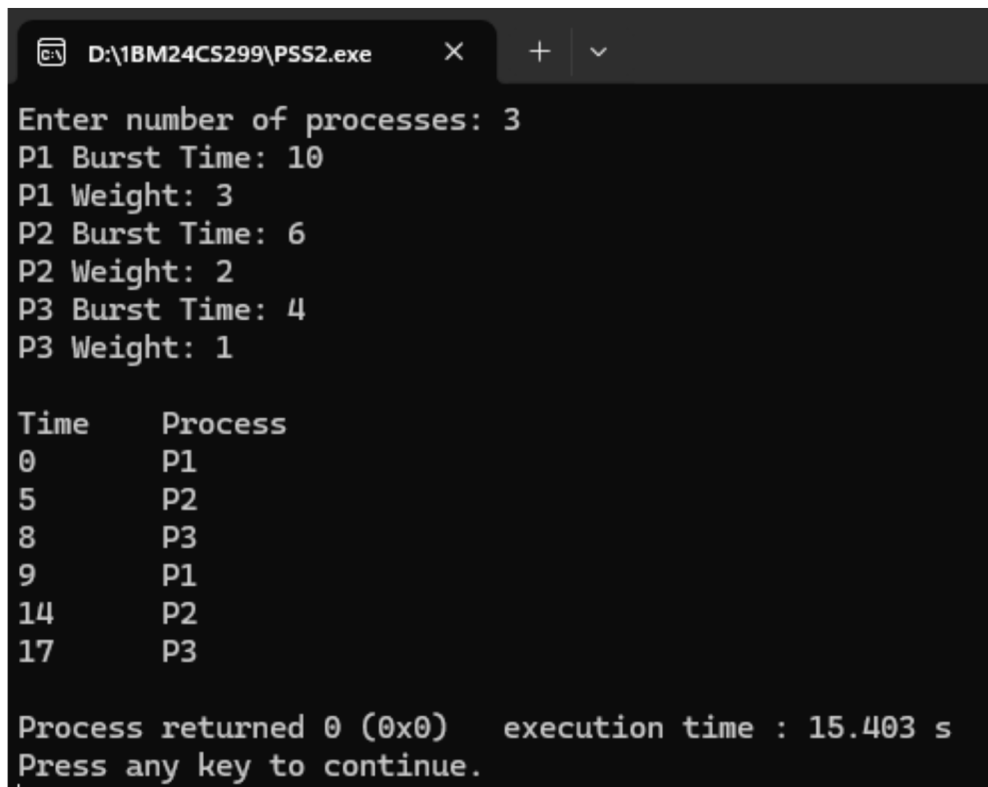
                if (slice > p[i].remaining) slice = p[i].remaining;
```

```

printf("%d\tP%d\n", time, p[i].id);
time += slice;
p[i].remaining -= slice;

if (p[i].remaining == 0) {
    completed++;
    total_weight -= p[i].weight;
}
}
}
}
return 0;
}

```



```

D:\1BM24CS299\P5S2.exe
Enter number of processes: 3
P1 Burst Time: 10
P1 Weight: 3
P2 Burst Time: 6
P2 Weight: 2
P3 Burst Time: 4
P3 Weight: 1

Time    Process
0       P1
5       P2
8       P3
9       P1
14      P2
17      P3

Process returned 0 (0x0)   execution time : 15.403 s
Press any key to continue.

```

13/3/26

Lab 6

1) Earliest Deadline First

```
#include <limits.h>
```

```
#include <stdio.h>
```

```
#define MAX 20
```

```
struct Task {
```

```
    int id;
```

```
    int at;
```

```
    int c;
```

```
    int D;
```

```
    int rem;
```

```
    int completed;
```

```
};
```

```
int main() {
```

```
    int n, i, time = 0, completed = 0;
```

```
    float U = 0;
```

```
    struct Task t[MAX];
```

```
    printf("Enter no. of tasks: ");
```

```
    scanf("%d", &n);
```

```
    for(i = 0; i < n; i++) {
```

```
        t[i].id = i + 1;
```

```
        printf("Task %d AT: ", i + 1);
```

```
        scanf("%d", &t[i].at);
```

```
        printf("Task %d Execution: ", i + 1);
```

```
        scanf("%d", &t[i].c);
```

```
        printf("Task %d Deadline: ", i + 1);
```

```
        scanf("%d", &t[i].D);
```



```
t[i].rem = t[i].c;
t[i].completed = 0;
```

```
U += (float) t[i].c / t[i].D;
```

```
pf ("Enter total time: ");
sf ("%d", &max_time);
```

```
if (U > 1) pf ("Not Feasible");
else printf ("Feasible");
```

```
printf ("\nTime\tTask\n");
```

```
while (time < max_time && completed < n)
```

```
{
```

```
int idx = -1;
```

```
int min_deadline = INT_MAX;
```

```
/* Find task with earliest deadline */
```

```
for (i = 0; i < n; i++) {
```

```
if (t[i].at <= time &&
    t[i].rem > 0)
```

```
if (t[i].D < min_deadline) {
    min_deadline = t[i].D;
```

```
idx = i;
}
```

```
}
```

```
}
```

```
if (idx == -1) {
```

```
pf ("%d\tIdle\n", time);
```

```
} else {
```

```
printf ("%d\tT.%d\n", time, t[idx].id);
```

```
t[idx].rem --;
```

```
if (t[idx].rem == 0) {
```

```
t[idx].completed = 1;
```

```
completed ++;
```

```
}
```



```

    }
    time ++;
    return 0;
}

```

Op: Enter no. of tasks: 3

Task 1 AT: 0

Task 1 Execution: 1

Task 1 Deadline: 4

Task 2 AT: 1

Task 2 Execution: 1

Task 2 Deadline: 3

Task 3 AT: 2

Task 3 Execution: 2

Task 3 Deadline: 6

Enter total simulation time: 10

Scheduling feasible

Time	Task
0	T1
1	T2
2	T3
3	T3
4	Idle
5	Idle
6	Idle
7	Idle
8	Idle
9	Idle

2) Proportional

#include

#define

#define

struct Pic

int

int

int

int

};

int main

int

int

int

struct

pf()

sf()

f

2) Proportional Share Scheduling

```
#include <stdio.h>
#define MAX 20
#define TO 10

struct Process {
    int id;
    int bt;
    int rem;
    int w; // weight (Priority)
};
```

```
int main() {
    int n, i, time = 0, completed = 0;
    int total_weight = 0;
    struct Process p[MAX];
```

```
    printf("Enter no. of processes : ");
    scanf("%d", &n);
```

```
    for(i = 0; i < n; i++) {
        p[i].id = i + 1;
        printf("P.%d BT: ", i + 1);
        scanf("%d", &p[i].bt);
```

```
        printf("P.%d weight: ", i + 1);
        scanf("%d", &p[i].w);
```

```
        p[i].rem = p[i].bt;
        total_weight += p[i].w;
```

```
    }
    while (completed < n) {
        if (total_weight == 0) break;
```


for (i = 0; i < n; i++) {

if (p[i].rem > 0) {

int slice = (p[i].weight /
total_weight) * 100;

if (slice <= 0) slice = 1;

if (slice > p[i].rem) {

slice = p[i].rem;

printf("%d\t %d\t %d\n", time, p[i].id,

time += slice;

p[i].rem -= slice;

if (p[i].rem == 0) {

completed++;

total_weight -= p[i].weight;

if (i == n-1) {

return 0;

}

Output:

Enter no. of processes : 3

P1 BT : 10

P1 weight : 3

P2 BT : 6

P2 weight : 2

P3 BT : 4

P3 weight : 1

Time	Process
0	P1
5	P2
8	P3
9	P1
14	P2
17	P3
18	

Sgt
13/1/20